

lab17(SSU)

October 29, 2025

1 Lab Assignment 17: Climate Change—Temperatures and Precipitation

In this project, you will investigate data on climate change, or the long-term shifts in temperatures and weather patterns!

```
[57]: !apt-get install texlive texlive-xetex texlive-latex-extra pandoc
      !pip install py pandoc
      !pip install datascience
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
texlive is already the newest version (2021.20220204-1).
texlive-latex-extra is already the newest version (2021.20220204-1).
texlive-xetex is already the newest version (2021.20220204-1).
0 upgraded, 0 newly installed, 0 to remove and 38 not upgraded.
Requirement already satisfied: py pandoc in /usr/local/lib/python3.12/dist-packages (1.15)
Requirement already satisfied: datascience in /usr/local/lib/python3.12/dist-packages (0.18.0)
Requirement already satisfied: folium>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from datascience) (0.20.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from datascience) (75.2.0)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from datascience) (3.10.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datascience) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from datascience) (1.16.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from datascience) (2.0.2)
Requirement already satisfied: ipython in /usr/local/lib/python3.12/dist-packages (from datascience) (7.34.0)
```

Requirement already satisfied: plotly in /usr/local/lib/python3.12/dist-packages (from datascience) (5.24.1)

Requirement already satisfied: branca in /usr/local/lib/python3.12/dist-packages (from datascience) (0.8.2)

Requirement already satisfied: jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (3.1.6)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (2.32.4)

Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium>=0.9.1->datascience) (2025.4.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (1.4.9)

Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (25.0)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0.0->datascience) (2.9.0.post0)

Requirement already satisfied: jedi>=0.16 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.19.2)

Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (4.4.2)

Requirement already satisfied: pickleshare in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.7.5)

Requirement already satisfied: traitlets>=4.2 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (5.7.1)

Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (3.0.52)

Requirement already satisfied: pygments in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (2.19.2)

Requirement already satisfied: backcall in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.2.0)

Requirement already satisfied: matplotlib-inline in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (0.2.1)

Requirement already satisfied: pexpect>4.3 in /usr/local/lib/python3.12/dist-packages (from ipython->datascience) (4.9.0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datascience) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datascience) (2025.2)

Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from plotly->datascience) (8.5.0)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in /usr/local/lib/python3.12/dist-packages (from jedi>=0.16->ipython->datascience) (0.8.5)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2>=2.9->folium>=0.9.1->datascience) (3.0.3)

Requirement already satisfied: ptyprocess>=0.5 in /usr/local/lib/python3.12/dist-packages (from pexpect>4.3->ipython->datascience) (0.7.0)

Requirement already satisfied: wcwidth in /usr/local/lib/python3.12/dist-packages (from prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0->ipython->datascience) (0.2.14)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->datascience) (1.17.0)

Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (3.4.4)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (3.11)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (2.5.0)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium>=0.9.1->datascience) (2025.10.5)

```
[58]: # Connect Google Drive to Colab so you can access your files
from google.colab import drive
drive.mount('/content/drive')

import os
os.chdir('/content/drive/MyDrive/Colab Notebooks/')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
[59]: # Run this cell to set up the notebook, but please don't change it.
from datascience import *
import numpy as np
```

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.style.use('fivethirtyeight')
np.set_printoptions(legacy='1.13')

import warnings
warnings.simplefilter('ignore')

import plotly.graph_objects as go
```

1.1 Part 1: Temperatures

In the following analysis, we will investigate one of the 21st century's most prominent issues: climate change. While the details of climate science are beyond the scope of this course, we can start to learn about climate change just by analyzing public records of different cities' temperature and precipitation over time.

We will analyze a collection of historical daily temperature and precipitation measurements from weather stations in 210 U.S. cities. The dataset was compiled by Yuchuan Lai and David Dzombak [1]; a description of the data from the original authors and the data itself is available [here](#).

[1] Lai, Yuchuan; Dzombak, David (2019): Compiled historical daily temperature and precipitation data for selected 210 U.S. cities. Carnegie Mellon University. Dataset.

1.1.1 Part 1, Section 1: Cities

Run the following cell to load information about the `cities` and preview the first few rows.

```
[60]: cities = Table.read_table('./DS/city_info.csv', index_col=0)
      cities.show(3)
```

<IPython.core.display.HTML object>

The `cities` table has one row per weather station and the following columns:

1. "Name": The name of the US city
2. "ID": The unique identifier for the US city
3. "Lat": The latitude of the US city (measured in degrees of latitude)
4. "Lon": The longitude of the US city (measured in degrees of longitude)
5. "Stn.Name": The name of the weather station in which the data was collected
6. "Stn.stDate": A string representing the date of the first recording at that particular station
7. "Stn.edDate": A string representing the date of the last recording at that particular station

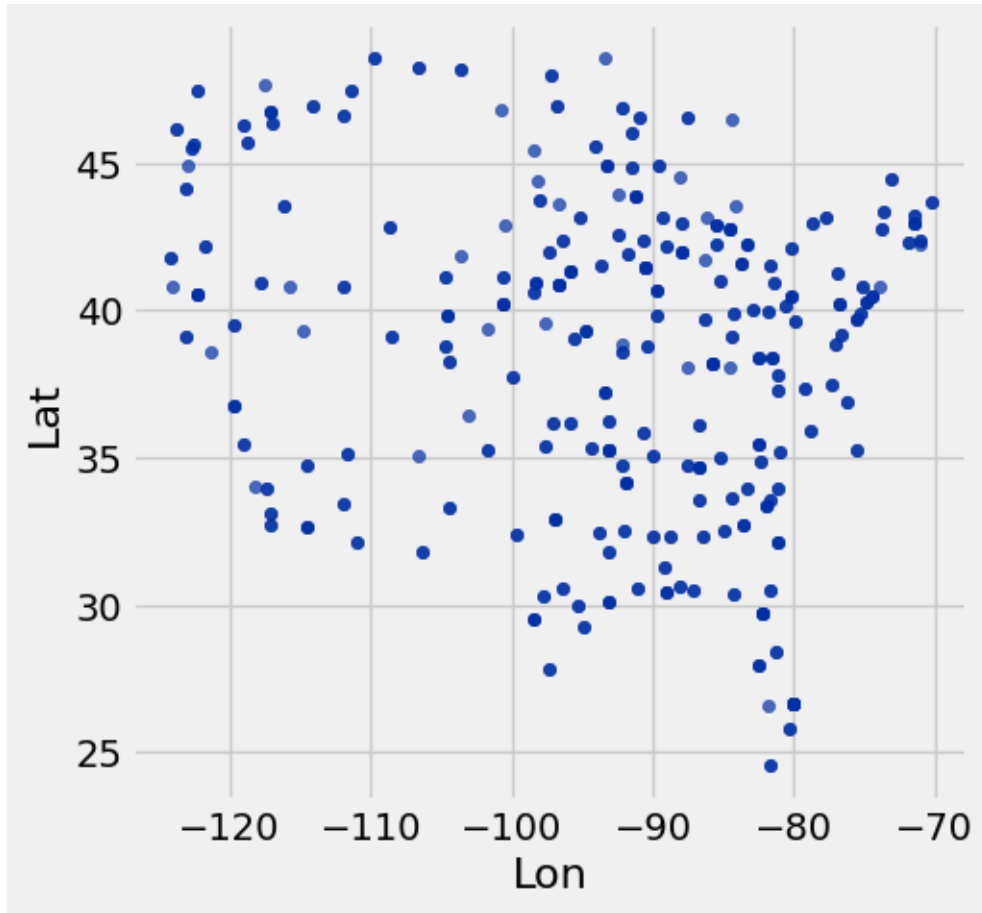
The data lists the weather stations at which temperature and precipitation data were collected. Note that although some cities have multiple weather stations, only one is collecting data for that city at any given point in time. Thus, we are able to just focus on the cities themselves.

Question 1.1.1: In the cell below, produce a scatter plot that plots the latitude and longitude of every city in the `cities` table so that the result places northern cities at the top and western cities at the left.

Note: It's okay to plot the same point multiple times!

Hint: A latitude is the set of horizontal lines that measures distances *north or south* of the equator. A longitude is the set of vertical lines that measures distances *east or west* of the prime meridian.

```
[61]: cities.scatter('Lon', 'Lat')
```



These cities are all within the continental U.S., and so the general shape of the U.S. should be visible in your plot. The shape will appear distorted compared to most maps for two reasons: the scatter plot is square even though the U.S. is wider than it is tall, and this scatter plot is an [equirectangular projection](#) of the spherical Earth. A geographical map of the same data uses the common [Pseudo-Mercator projection](#).

```
[62]: # Just run this cell
fig = go.Figure(data=go.Scattergeo(
    lon = cities.column('Lon'),
    lat = cities.column('Lat'),
    text = cities.column('Name'),
    mode = 'markers'))
```

```
fig.update_layout(geo_scope='usa', width=800, height=500, margin={'r': 0, 't': 0, 'l': 0, 'b': 0})
fig.show()
```

Question 1.1.2 Does it appear that these city locations are sampled uniformly at random from all the locations in the U.S.? Why or why not?

Looking at the scatterplot, cities appear to be spread across the United States, but they appear to be more concentrated in certain areas. Therefore, the location of cities can be a valuable resource for the United States. This could potentially have an impact.

Question 1.1.3: Assign `num_unique_cities` to the number of unique cities that appear in the `cities` table.

```
[63]: num_unique_cities = cities.group('Name').num_rows

# Do not change this line
print(f"There are {num_unique_cities} unique cities that appear within our dataset.")
```

There are 210 unique cities that appear within our dataset.

In order to investigate further, it will be helpful to determine what region of the United States each city was located in: Northeast, Northwest, Southeast, or Southwest. For our purposes, we will be using the following geographical boundaries:

1. A station is located in the "Northeast" region if its latitude is above or equal to 40 degrees and its longitude is greater than or equal to -100 degrees.
2. A station is located in the "Northwest" region if its latitude is above or equal to 40 degrees and its longitude is less than -100 degrees.
3. A station is located in the "Southeast" region if its latitude is below 40 degrees and its longitude is greater than or equal to -100 degrees.
4. A station is located in the "Southwest" region if its latitude is below 40 degrees and its longitude is less than -100 degrees.

Question 1.1.4: Define the `coordinates_to_region` function below. It should take in two arguments, a city's latitude (`lat`) and longitude (`lon`) coordinates, and output a string representing the region it is located in.

```
[64]: def coordinates_to_region(lat, lon):
    if lat >= 40 and lon >= -100:
        return "Northeast"
    elif lat >= 40 and lon < -100:
        return "Northwest"
    elif lat < 40 and lon >= -100:
        return "Southeast"
    elif lat < 40 and lon < -100:
        return "Southwest"
```

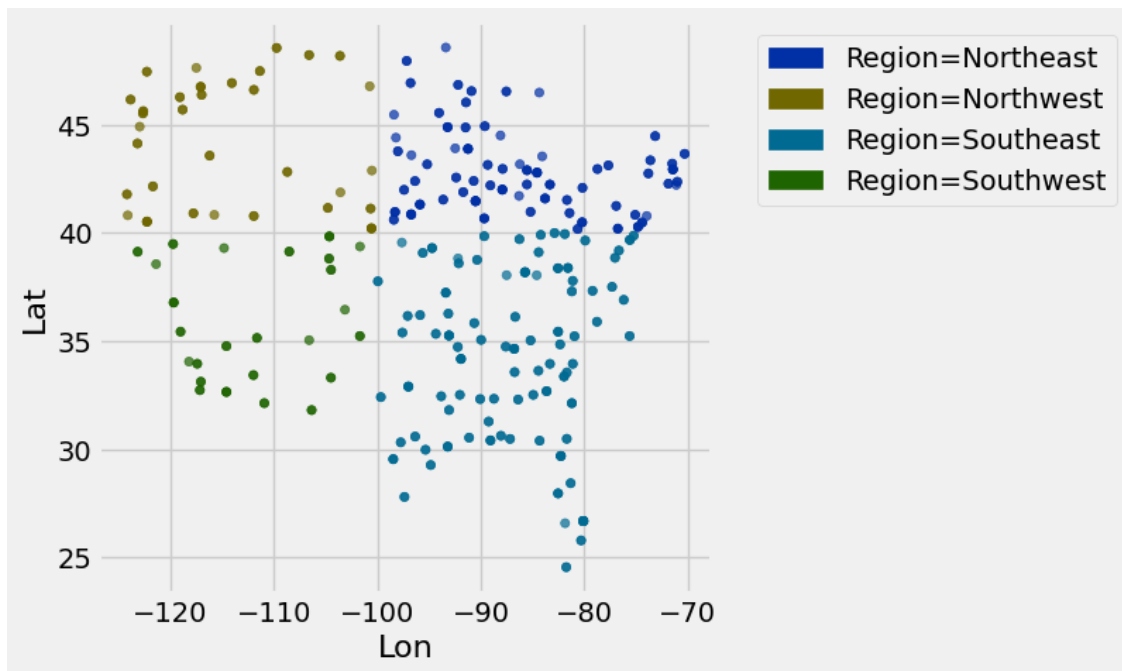
Question 1.1.5: Add a new column in `cities` labeled `Region` that contains the region in which the city is located. For full credit, you must use the `coordinates_to_region` function you defined rather than reimplementing its logic.

```
[65]: regions_array = cities.apply(coordinates_to_region, 'Lat', 'Lon')
      cities = cities.with_column('Region', regions_array)
      cities.show(5)
```

<IPython.core.display.HTML object>

To confirm that you've defined your `coordinates_to_region` function correctly and successfully added the `Region` column to the `cities` table, run the following cell. Each region should have a different color in the result.

```
[66]: # Just run this cell
      cities.scatter("Lon", "Lat", group="Region")
```



Challenge Question 1.1.6 (OPTIONAL, ungraded): Create a new table called `cities_nearest`. It should contain the same columns as the `cities` table and an additional column called "Nearest" that contains the **name of the nearest city** that is in a different region from the city described by the row.

To approximate the distance between two cities, take the square root of the sum of the squared difference between their latitudes and the square difference between their longitudes. Don't use a `for` statement; instead, use the `apply` method and array arithmetic.

Hint: We have defined a `distance` function for you, which can be called on numbers `lat0` and `lon0` and arrays `lat1` and `lon1`.

[66]:

```
[67]: def distance(lat0, lon0, lat1, lon1):
    "Approximate the distance between point (lat0, lon0) and (lat1, lon1) pairs
    ↪in the arrays."
    return np.sqrt((lat0 - lat1) * (lat0 - lat1) + (lon0 - lon1) * (lon0 -
    ↪lon1))

# BEGIN SOLUTION
def nearest(name):
    row = cities.where('Name', name).row(0)
    others = cities.where('Region', are.not_equal_to(row.item('Region')))
    distances = distance(row.item('Lat'), row.item('Lon'),
        others.column('Lat'), others.column('Lon'))
    return others.with_column('dist', distances).sort('dist').row(0).
    ↪item('Name')

nearest_array = cities.apply(nearest, "Name")
# END SOLUTION

cities_nearest = cities.with_column("Nearest", nearest_array) # SOLUTION
cities_nearest.show(5)
```

<IPython.core.display.HTML object>

1.1.2 Part 1, Section 2: Welcome to Phoenix, Arizona

Each city has a different CSV file full of daily temperature and precipitation measurements. The file for Phoenix, Arizona is included with this project as `phoenix.csv`. The files for other cities can be downloaded [here](#) by matching them to the ID of the city in the `cities` table.

Since Phoenix is located on the upper edge of the Sonoran Desert, it has some impressive temperatures.

Run the following cell to load in the `phoenix` table. It has one row per day and the following columns:

1. "Date": The date (a string) representing the date of the recording in **YYYY-MM-DD** format
2. "tmax": The maximum temperature for the day (°F)
3. "tmin": The minimum temperature for the day (°F)
4. "prcp": The recorded precipitation for the day (inches)

```
[68]: phoenix = Table.read_table("./DS/phoenix.csv", index_col=0)
      phoenix.show(3)
```

<IPython.core.display.HTML object>

Question 1.2.1: Assign the variable `largest_2010_range_date` to the date of the **largest temperature range** in Phoenix, Arizona for any day between January 1st, 2010 and December 31st,

2010. To get started, use the variable `phoenix_with_ranges_2010` to filter the phoenix table to days in 2010 with an additional column corresponding to the temperature range for that day.

Your answer should be a string in the “YYYY-MM-DD” format. Feel free to use as many lines as you need. A temperature range is calculated as the difference between the max and min temperatures for the day.

Hint: To limit the values in a column to only those that *contain* a certain string, pick the right `are.` predicate from the [Python Reference Sheet](#).

Note: Do **not** re-assign the `phoenix` variable; please use the `phoenix_with_ranges_2010` variable instead.

```
[69]: phoenix_with_ranges = phoenix.with_column('range', phoenix.column('tmax') -  
      ↪ phoenix.column('tmin'))  
      phoenix_with_ranges_2010 = phoenix_with_ranges.where('Date', are.  
      ↪ containing('2010'))  
      largest_2010_range_date = phoenix_with_ranges_2010.sort('range',  
      ↪ descending=True).column('Date').item(0)  
      largest_2010_range_date
```

```
[69]: '2010-06-24'
```

We can look back to our `phoenix` table to check the temperature readings for our `largest_2010_range_date` to see if anything special is going on. Run the cell below to find the row of the phoenix table that corresponds to the date we found above.

```
[70]: # Just run this cell  
      phoenix.where("Date", largest_2010_range_date)
```

```
[70]: Date          | tmax | tmin | prcp  
      2010-06-24 | 113  | 79   | 0
```

ZOO WEE MAMA! Look at the maximum temperature for that day. That’s hot.

The function `extract_year_from_date` takes a date string in the **YYYY-MM-DD** format and returns an integer representing the **year**. The function `extract_month_from_date` takes a date string and returns a string describing the month. Run this cell, but you do not need to understand how this code works or edit it.

```
[71]: # Just run this cell  
      import calendar  
  
      def extract_year_from_date(date):  
          """Returns an integer corresponding to the year of the input string's date.  
          ↪ """  
          return int(date[:4])  
  
      def extract_month_from_date(date):  
          "Return an abbreviation of the name of the month for a string's date."
```

```

month = date[5:7]
return f'{month} ({calendar.month_abbr[int(date[5:7])])}'

# Example
print('2022-04-01 has year', extract_year_from_date('2022-04-01'),
      'and month', extract_month_from_date('2022-04-01'))

```

2022-04-01 has year 2022 and month 04 (Apr)

Question 1.2.2: Add two new columns called **Year** and **Month** to the **phoenix** table that contain the year as an **integer** and the month as a **string** (such as "04 (Apr)") for each day, respectively.

Note: The functions above may be helpful!

```

[72]: years_array = phoenix.apply(extract_year_from_date, 'Date')
      months_array = phoenix.apply(extract_month_from_date, 'Date')
      phoenix = phoenix.with_columns('Year', years_array, 'Month', months_array)
      phoenix.show(5)

```

<IPython.core.display.HTML object>

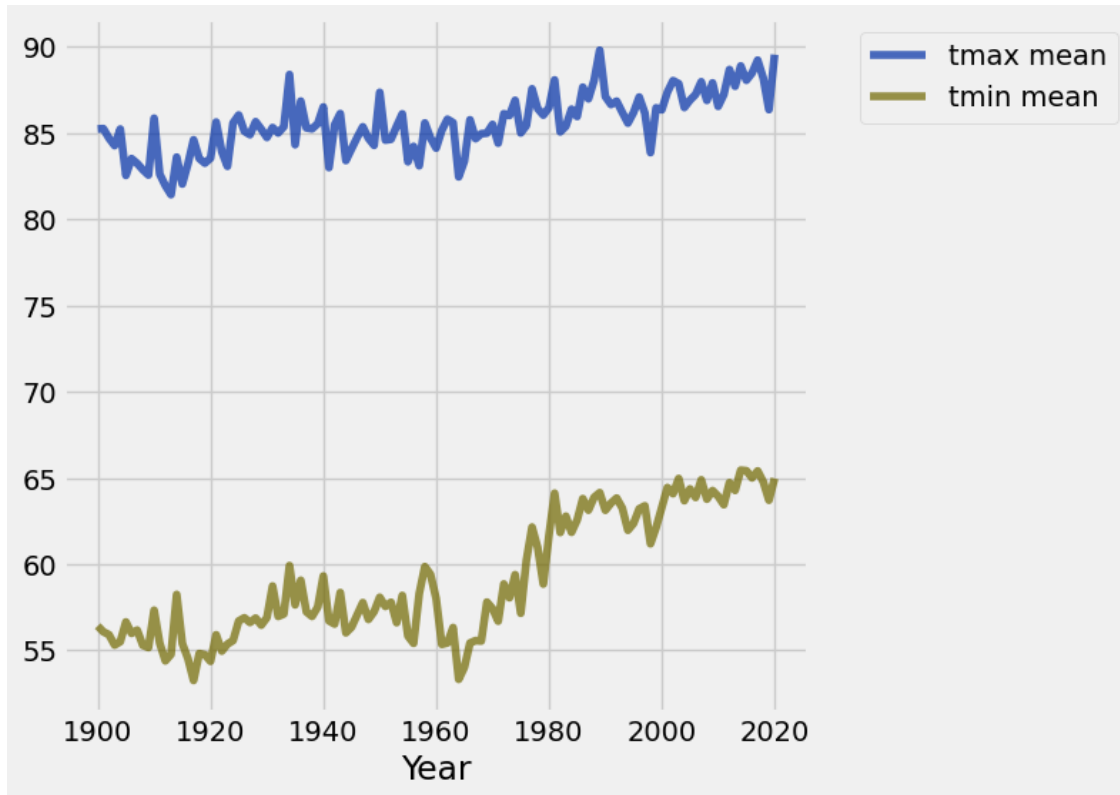
Question 1.2.3: Using the **phoenix** table, create an overlaid line plot of the **average maximum temperature** and **average minimum temperature** for each year between 1900 and 2020 (inclusive).

Hint: To draw a line plot with more than one line, call **plot** on the column label of the x-axis values and all other columns will be treated as y-axis values.

```

[73]: avg_temps_by_year = phoenix.group('Year', np.mean).select('Year', 'tmax mean',
    ↪ 'tmin mean')
      avg_temps_1900To2020 = avg_temps_by_year.where('Year', are.
    ↪ between_or_equal_to(1900, 2020))
      avg_temps_1900To2020.plot('Year')

```



Question 1.2.4: Although still hotly debated (pun intended), many climate scientists agree that the effects of climate change began to surface in the early 1960s as a result of elevated levels of greenhouse gas emissions. How does the graph you produced in Question 1.2.3 support the claim that modern-day global warming began in the early 1960s?

Looking at the graph generated in question 1.2.3, both the average annual maximum temperature and climate in Phoenix show a notable trend starting in the 1960s. Before the 1960s, there was an internal trend of change. However, since the 1960s, more eruptions have occurred in the northern region. These observations may provide evidence, as many argue, that human activity has begun to impact the Earth since the 1960s.

Averaging temperatures across an entire year can obscure some effects of climate change. For example, if summers get hotter but winters get colder, the annual average may not change much. Let's investigate how average **monthly** maximum temperatures have changed over time in Phoenix.

Question 1.2.5: Create a `monthly_increases` table with one row per month and the following four columns in order: 1. "Month": The month (such as "02 (Feb)") 2. "Past": The average max temperature in that month from 1900-1960 (both ends inclusive) 3. "Present": The average max temperature in that month from 2019-2021 (both ends inclusive) 4. "Increase": The difference between the present and past average max temperatures in that month

First make a copy of the `phoenix` table and add a new column containing the corresponding **period** for each row. You may find the `period` function helpful. Then, use this new table to construct `monthly_increases`. Feel free to use as many lines as you need.

Hint: What table method can we use to get each **unique value** as its own column?

Note: Please do **not** re-assign the phoenix variable!

```
[74]: def period(year):
    "Output if a year is in the Past, Present, or Other."
    if 1900 <= year <= 1960:
        return "Past"
    elif 2019 <= year <= 2021:
        return "Present"
    else:
        return "Other"

phoenix_with_period = phoenix.with_column('Period', phoenix.apply(period,
    ↪ 'Year'))

monthly_avg_tmax = phoenix_with_period.where('Period', are.
    ↪ not_equal_to('Other')).group(['Month', 'Period'], np.mean).select('Month',
    ↪ 'Period', 'tmax mean')

past_temps = monthly_avg_tmax.where('Period', 'Past').select('Month', 'tmax_
    ↪ mean').reabeled('tmax mean', 'Past')
present_temps = monthly_avg_tmax.where('Period', 'Present').select('Month',
    ↪ 'tmax mean').reabeled('tmax mean', 'Present')

monthly_increases = past_temps.join('Month', present_temps)
monthly_increases = monthly_increases.with_columns('Increase',
    ↪ monthly_increases.column('Present') - monthly_increases.column('Past'))
monthly_increases.show()
```

<IPython.core.display.HTML object>

1.2 Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a pdf file for you to submit. **Please save before exporting!**

```
[75]: # should change the directory and file name matching to yours
!jupyter nbconvert './DS/lab17(SSU).ipynb' --to pdf
```

```
[NbConvertApp] Converting notebook ./DS/lab17(SSU).ipynb to pdf
/usr/local/share/jupyter/nbconvert/templates/latex/display_priority.j2:32:
UserWarning: Your element with mimetype(s) dict_keys(['text/html']) is not able
to be represented.
((*- endblock -*))
[NbConvertApp] Support files will be in lab17(SSU)_files/
[NbConvertApp] Making directory ./lab17(SSU)_files
[NbConvertApp] Writing 89486 bytes to notebook.tex
```

```
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no
citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 221472 bytes to DS/lab17(SSU).pdf
```